
DATA ENGINEERING COURSE

Python Modules & Code Organisation

Class Notes

Instructor: Tanishq Tyagi

Topics Covered

- Part 1: What is a Module
- Part 2: Importing Modules (import, from...import, aliasing)
- Part 3: Creating Custom Modules
- Part 4: The if __name__ == "__main__" Concept
- Part 5: Importance of Project Structure in Data Engineering

Table of Contents

1. Session Overview
2. Learning Objectives
3. Part 1: What is a Module
4. Part 2: Importing Modules
 - 4.1 import module
 - 4.2 from module import function
 - 4.3 Aliasing with as
5. Part 3: Creating Custom Modules
6. Part 4: The if `__name__ == "__main__"` Concept
7. Part 5: Project Structure in Data Engineering
8. Practical Examples
9. Real-World Data Engineering Perspective
10. Summary & Quick Revision Sheet
11. General Interview Questions
12. Data Engineer Interview Preparation

1 | Session Overview

In this session we explored one of the most critical yet often overlooked concepts in Python programming: **Modules and Code Organisation**. As programs grow in size and complexity, writing all code in a single file becomes unmanageable, error-prone, and difficult to maintain. Python's module system provides an elegant solution to this problem.

We covered what a module is, how to import code from other files using three distinct methods, how to build reusable custom modules, the critical `if __name__ == "__main__"` guard, and how professional data engineers structure their projects for production environments.

2 | Learning Objectives

By the end of this session, students will be able to:

- Define what a Python module is and explain why modules are essential in real-world programming.
- Import modules using three different methods: `import module`, `from module import function`, and aliasing with `as`.
- Create custom reusable Python modules and share functionality across multiple files.
- Explain the purpose and behaviour of the `if __name__ == "__main__"` guard.
- Design a professional project folder structure following data engineering best practices.
- Connect module concepts to ETL pipelines, PySpark jobs, and production data systems.
- Answer common interview questions on Python modules and project structure.

3 | Part 1: What is a Module

Definition

A **module** is simply a Python file (with a `.py` extension) that contains Python code — functions, classes, variables, or any executable statements. Modules allow you to logically organise your code into separate files and reuse that code across multiple programs.

■ Analogy

Think of a module as a toolbox. Instead of carrying every tool in your hands, you store them neatly in a toolbox and pick up only the tool you need when you need it.

Why Modules Are Important

- **Reusability:** Write a function once, use it in many files without copy-pasting.
- **Maintainability:** Changes to a function in one file automatically propagate everywhere it is used.
- **Readability:** Smaller, focused files are far easier to understand than a single 2,000-line script.
- **Collaboration:** Teams can work on separate modules independently without merge conflicts.
- **Namespacing:** Modules create separate namespaces, preventing naming collisions between functions.
- **Testing:** Individual modules can be tested in isolation (unit testing).

Real-World Application

Every serious Python project — whether it is a web application, a machine learning model, or a data pipeline — is built using modules. NumPy, Pandas, PySpark, and Airflow are all module libraries. When you run `import pandas as pd`, you are importing a module.

Creating Your First Module

A module is just a `.py` file. For example:

```
# File: math_util.py

def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
        raise ValueError('Division by zero is not allowed.')
    return a / b
```

math_util.py — A simple custom utility module

■ Key Takeaway

The file `math_util.py` is now a module. It can be imported into any other Python file in the same project.

4 | Part 2: Importing Modules

Python provides three main ways to import code from a module. Each has its own use case, advantages, and trade-offs. Understanding when to use which method is an important skill.

4.1 Method 1: import module

The simplest form of import. This loads the entire module into memory and makes it accessible through the module name as a prefix.

```
# main.py

import math_util

result = math_util.add(2, 3)
print(f'Addition: {result}')           # Output: Addition: 5

result = math_util.subtract(10, 4)
print(f'Subtraction: {result}')        # Output: Subtraction: 6

result = math_util.multiply(3, 7)
print(f'Multiplication: {result}')     # Output: Multiplication: 21

result = math_util.divide(15, 3)
print(f'Division: {result}')           # Output: Division: 5.0
```

Method 1: Importing the full module

How It Works — Line by Line

- `import math_util` — Python searches for a file named `math_util.py` in the current directory and `sys.path`. It executes the file and stores the resulting module object under the name `math_util`.
- `math_util.add(2, 3)` — We access the `add` function using dot notation. The module name acts as a namespace.

■ Trade-offs

Advantage: You always know where a function comes from (e.g., `math_util.add` is clearly from `math_util`).

Disadvantage: More verbose — you must type the module name every time.

Common Mistake

■ Common Mistake

Forgetting the module prefix and calling `add(2, 3)` directly after `import math_util` will raise a `NameError`. With this method you must always use `math_util.add(2, 3)`.

4.2 Method 2: from module import function

This method imports specific names (functions, classes, or variables) directly into the current namespace. You can then use them without any prefix.

```
# main.py

from math_util import divide, add

print(divide(10, 2))    # Output: 5.0
print(add(2, 3))        # Output: 5
```

Method 2: Importing specific names from a module

How It Works

Only `divide` and `add` are imported — not `subtract` or `multiply`. These two names are added directly to the local namespace, so no prefix is needed.

■ Trade-offs

Advantage: Cleaner, more concise code — no prefix required.

Disadvantage: If two modules define a function with the same name, the second import will silently overwrite the first, causing hard-to-find bugs.

Importing Everything with *

It is also possible to write `from math_util import *`, which imports every public name from the module. This is **strongly discouraged** in production code because it pollutes the namespace and makes it impossible to trace where a function came from.

■ Best Practice Warning

Never use `from module import *` in production or team projects. It makes code harder to read and debug.

4.3 Method 3: Aliasing with as

Aliasing allows you to import a module (or a specific name) under a shorter or more convenient name. This is extremely common in data engineering and data science.

```
# main.py

import math_util as mu

print(mu.add(2, 3))      # Output: 5
print(mu.subtract(10, 4)) # Output: 6
print(mu.multiply(3, 7)) # Output: 21
print(mu.divide(15, 3))  # Output: 5.0
```

Method 3: Aliasing a module with 'as'

Why Use Aliasing?

- **Shorter code:** `mu.add()` instead of `math_util.add()`.
- **Industry convention:** `import pandas as pd`, `import numpy as np`, `import pyspark.sql.functions as F` — these aliases are universally recognised.
- **Readability:** Aliases communicate intent and are often standard across teams.
- **Conflict resolution:** If two modules have the same name, aliasing one of them avoids confusion.

Industry Examples of Aliasing

Library	Standard Alias	Domain
pandas	pd	Data Science / Engineering
numpy	np	Scientific Computing
pyspark.sql.functions	F	Big Data / Spark
matplotlib.pyplot	plt	Data Visualisation
tensorflow	tf	Machine Learning
sqlalchemy	sa	Database ORM

Aliasing a Specific Function

```
from math_util import multiply as mul

print(mul(4, 5))    # Output: 20
```

Aliasing a specific function

5 | Part 3: Creating Custom Modules

Creating a custom module is as simple as saving your functions in a `.py` file. However, professional modules follow conventions that make them robust, testable, and reusable.

Complete Example: `math_util.py`

```
"""
math_util.py
Utility module providing basic mathematical operations.
Part of the Data Engineering Course project.
"""

def add(a, b):
    """Return the sum of a and b."""
    return a + b

def subtract(a, b):
    """Return the difference of a and b."""
    return a - b

def multiply(a, b):
    """Return the product of a and b."""
    return a * b

def divide(a, b):
    """Return the quotient of a and b. Raises ValueError for division by zero."""
    if b == 0:
        raise ValueError('Cannot divide by zero.')
    return a / b
```

Professional `math_util.py` module with docstrings

Best Practices for Writing Modules

- **Add a module-level docstring** describing the module's purpose.
- **Add docstrings to every function** explaining what it does, its parameters, and return value.
- **Use meaningful function names** that clearly describe the operation.
- **Validate inputs** and raise meaningful exceptions (e.g., `ValueError`) instead of letting Python crash silently.
- **Keep modules focused** — a module should do one thing well (Single Responsibility Principle).
- **Avoid side effects** at the module level — code that runs on import should only define functions and constants, not print output or modify global state.

Using the Module

```
# main.py

import math_util as mu

print(mu.add(10, 5))      # Output: 15
print(mu.subtract(10, 5)) # Output: 5
print(mu.multiply(10, 5)) # Output: 50
print(mu.divide(10, 5))   # Output: 2.0

# Error handling
try:
    print(mu.divide(10, 0))
except ValueError as e:
    print(f'Error: {e}')   # Output: Error: Cannot divide by zero.
```

Importing and using the custom math_util module

6 | Part 4: The if `__name__ == "__main__"` Concept

What is `__name__`?

Every Python file has a built-in variable called `__name__`. Its value depends on *how* the file is being run:

Scenario	<code>__name__</code> value
File is run directly (python main.py)	<code>"__main__"</code>
File is imported into another file	The module name (e.g., <code>"main"</code>)

Demonstration

```
# main.py

if __name__ == "__main__":
    print(__name__)      # Prints: __main__
    print("Hello main!")
```

From the class notes: `__name__` guard in main.py

When you run `python main.py` directly, Python sets `__name__` to the string `"__main__"`. The condition is `True` and the code inside runs.

When another script does `import main`, Python sets `__name__` to `"main"` (the filename without `.py`). The condition is `False` and the code inside does NOT run.

Why This Matters

Without this guard, any code at the module level runs whenever the file is imported — even if you only wanted to use the functions defined in it. This causes unintended side effects such as printing output, making database connections, or launching processes during import.

Real-World Pattern

```
# etl_pipeline.py

def extract():
    print('Extracting data...')
    return [{'id': 1, 'value': 100}, {'id': 2, 'value': 200}]

def transform(data):
    return [{'id': r['id'], 'value': r['value'] * 1.1} for r in data]

def load(data):
    print(f'Loading {len(data)} records...')

if __name__ == "__main__":
```

```
# This block only runs when we execute: python etl_pipeline.py
# It will NOT run when another module imports etl_pipeline
raw_data = extract()
transformed = transform(raw_data)
load(transformed)
```

Production ETL pipeline using `__name__` guard

■ Key Takeaway

The `if __name__ == "__main__"` guard is one of the most important Python idioms. You will see it in virtually every professional Python project. Always use it to protect your script's entry point.

7 | Part 5: Project Structure in Data Engineering

As projects grow, keeping all code in a single file or flat directory becomes unmanageable. Professional data engineers organise their code into structured packages with clear separation of concerns.

Why Project Structure Matters

- **Scalability:** A well-structured project can grow from 100 to 100,000 lines without becoming chaotic.
- **Maintainability:** New team members can navigate the codebase without a guide.
- **Testability:** Separate modules can be unit-tested independently.
- **CI/CD Compatibility:** Tools like GitHub Actions, Jenkins, and Docker rely on predictable file structures.
- **Team Collaboration:** Different engineers can own different modules with minimal conflicts.

Class Example Structure (from main.py)

```
Day_13_Modules_Code_Organisation/  
  
■■■ main.py                # Entry point  
  
■■■ utils/  
    ■■■ math_util.py       # Reusable math utilities
```

Simple module structure from today's class

Professional Data Engineering Project Structure

```
my_data_pipeline/  
  
■■■ main.py                # Pipeline entry point  
■■■ requirements.txt       # Python dependencies  
■■■ README.md              # Project documentation  
■■■ config/  
■   ■■■ __init__.py  
■   ■■■ settings.py        # Environment configs, constants  
■■■ ingestion/  
■   ■■■ __init__.py  
■   ■■■ api_reader.py      # Read from REST APIs  
■   ■■■ db_reader.py       # Read from databases  
■■■ transformation/  
■   ■■■ __init__.py  
■   ■■■ cleaner.py         # Data cleaning functions  
■   ■■■ aggregator.py      # Aggregation logic  
■■■ loading/  
■   ■■■ __init__.py  
■   ■■■ warehouse_writer.py # Write to data warehouse
```

```
■■■ utils/
■   ■■■ __init__.py
■   ■■■ math_util.py      # Math helper functions
■   ■■■ logger.py         # Logging utilities
■■■ tests/
    ■■■ test_cleaner.py
    ■■■ test_aggregator.py
```

Professional ETL project structure

What is `__init__.py`?

An `__init__.py` file marks a directory as a Python **package**. It can be empty or contain initialisation code. Without it, Python (in older versions) would not recognise the directory as importable. In Python 3, it is optional for simple cases but is still considered best practice for clarity.

```
# Importing from a sub-package
from utils.math_util import add, divide

# Or with alias
import utils.math_util as mu
```

Importing from a package structure

■ From the Class Notes

In the class example, the import path is `Day_13_Modules_Code_Organisation.utils.math_util` — reflecting the exact folder structure on disk.

8 | Practical Examples

Example 1: Full Calculator Using a Custom Module

Step 1: Create math_util.py

```
# utils/math_util.py

def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
        raise ValueError('Cannot divide by zero.')
    return a / b
```

Step 2: Create main.py (User-Driven Calculator)

```
# main.py

import utils.math_util as mu

if __name__ == "__main__":
    print('=== Python Calculator ===')
    num1 = float(input('Enter first number: '))
    num2 = float(input('Enter second number: '))
    op = input('Enter operation (+, -, *, /): ')

    if op == '+':
        print(f'Result: {mu.add(num1, num2)}')
    elif op == '-':
        print(f'Result: {mu.subtract(num1, num2)}')
    elif op == '*':
        print(f'Result: {mu.multiply(num1, num2)}')
    elif op == '/':
        try:
            print(f'Result: {mu.divide(num1, num2)}')
        except ValueError as e:
            print(e)
```

```
        print(f'Error: {e}')
    else:
        print('Invalid operation.')
```

User-driven calculator using math_util module

Example Output

```
=== Python Calculator ===
Enter first number: 10
Enter second number: 4
Enter operation (+, -, *, /): /
Result: 2.5
```

Sample output

Example 2: String Analyser with string_util Module

string_util.py

```
# string_util.py

def count_words(text):
    """Return the number of words in text."""
    return len(text.split())

def count_vowels(text):
    """Return the count of vowels in text."""
    return sum(1 for ch in text.lower() if ch in 'aeiou')

def reverse_string(text):
    """Return the reversed version of text."""
    return text[::-1]

def is_palindrome(text):
    """Return True if text is a palindrome (case-insensitive)."""
    cleaned = text.lower().replace(' ', '')
    return cleaned == cleaned[::-1]

def most_common_char(text):
    """Return the most frequently occurring character in text."""
    freq = {}
    for ch in text:
        freq[ch] = freq.get(ch, 0) + 1
    return max(freq, key=freq.get)
```


string_util.py — String analysis utility module

string_analyser.py

```
# string_analyser.py
import string_util as su

if __name__ == "__main__":
    text = input('Enter a string: ')
    print(f'Word count      : {su.count_words(text)}')
    print(f'Vowel count     : {su.count_vowels(text)}')
    print(f'Reversed          : {su.reverse_string(text)}')
    print(f'Is palindrome   : {su.is_palindrome(text)}')
    print(f'Most common ch : {su.most_common_char(text)}')
```

string_analyser.py — Main program using string_util

Example Output

```
Enter a string: racecar
Word count      : 1
Vowel count     : 3
Reversed        : racecar
Is palindrome   : True
Most common ch : r
```

9 | Real-World Data Engineering Perspective

How Data Engineers Use Modules Daily

Modules are the backbone of every production data pipeline. Here is how the concepts from today's session map directly to real data engineering work:

ETL Pipelines:

Each stage — Extract, Transform, Load — is typically a separate module. The extract module connects to APIs or databases, the transform module applies business logic, and the load module writes to the data warehouse. This separation allows each stage to be independently tested, deployed, and scaled.

PySpark Jobs:

Large-scale Spark jobs import shared utility modules for schema definitions, transformation functions, and logging. `import pyspark.sql.functions as F` is one of the most common imports in the entire data engineering ecosystem.

Airflow DAGs:

Apache Airflow DAGs (Directed Acyclic Graphs) import Python modules to define task logic. Each operator imports from shared utility packages, keeping DAG files clean and readable.

Config Management:

Production pipelines separate configuration (database URLs, API keys, batch sizes) into a dedicated `config/settings.py` module. This allows environment-specific configuration without modifying core business logic.

Unit Testing:

Individual modules are tested independently using `pytest` or `unittest`. The `if __name__ == "__main__":` guard ensures that test files do not accidentally trigger pipeline execution when imported by the test runner.

Scalability Considerations

Well-structured module systems scale naturally. When a team grows from 2 to 20 engineers, each engineer can own specific modules without stepping on others' work. Continuous Integration pipelines can run tests for specific modules when only those files change, dramatically reducing build times.

10 | Summary & Quick Revision Sheet

Summary of Key Concepts

Concept	Description
Module	A .py file containing reusable Python code (functions, classes, variables).
import module	Loads entire module; access via module.function() prefix.
from module import fn	Imports specific names directly into current namespace.
import module as alias	Loads module under a shorter alias; standard in data engineering.
__name__	Built-in variable; equals "__main__" when file runs directly.
if __name__ == '__main__':	Guards entry-point code from running on import.
Package	A directory containing __init__.py and multiple modules.
Project Structure	Organise code into logical folders for scalability and maintainability.

Syntax Cheat Sheet

```
# Method 1: Import full module
import math_util
math_util.add(2, 3)

# Method 2: Import specific function
from math_util import add, divide
add(2, 3)

# Method 3: Alias
import math_util as mu
mu.add(2, 3)

# Entry point guard
if __name__ == "__main__":
    main()

# Import from sub-package
from utils.math_util import add
import utils.math_util as mu
```

Quick syntax reference

Common Mistakes to Avoid

- Using `from module import *` in production code.

- Forgetting the module prefix when using `import module`.
- Placing executable code at module level without the `if __name__ == "__main__":` guard.
- Creating very large modules instead of splitting into focused sub-modules.
- Circular imports (module A imports from B, and B imports from A).

11 | General Interview Questions

A. Beginner Level

Q1. What is a Python module?

Answer: A module is a .py file that contains Python code — functions, classes, or variables — that can be imported and reused in other Python programs. For example, `math_util.py` is a module.

Q2. How do you import a module in Python?

Answer: Using the import statement: `import math_util`. After this, you access functions with the module name as prefix: `math_util.add(2, 3)`.

Q3. What is the difference between 'import module' and 'from module import function'?

Answer: 'import module' loads the full module and requires a prefix. 'from module import function' imports specific names directly into the current namespace, requiring no prefix.

Q4. What is aliasing in Python imports?

Answer: Aliasing assigns a shorter name to a module using 'as': `import math_util as mu`. This is common in data engineering (e.g., `import pandas as pd`).

Q5. Can you import multiple functions from a module at once?

Answer: Yes: `from math_util import add, subtract, multiply`. All listed names are imported directly.

Q6. What is the `__name__` variable in Python?

Answer: `__name__` is a built-in variable present in every Python file. When a file runs directly, `__name__` equals `"__main__"`. When imported, it equals the module's filename (without .py).

Q7. What does `if __name__ == "__main__":` do?

Answer: It ensures the code inside the block only runs when the file is executed directly, not when it is imported. This protects entry-point logic from running during imports.

Q8. How do you create a custom module?

Answer: Simply create a .py file with your functions and save it. For example, save `add()` and `subtract()` in `math_util.py`. Any other file can then import it.

Q9. What is a Python package?

Answer: A package is a directory containing an `__init__.py` file and one or more module files. It allows hierarchical organisation of modules.

Q10. Why is it bad practice to use 'from module import *'?

Answer: It imports all names into the current namespace, making it impossible to know where a function came from, risking name collisions and making code harder to debug.

B. Intermediate Level

Q1. What is the difference between a module and a package?

Answer: A module is a single .py file. A package is a directory of modules marked by an `__init__.py` file, allowing imports like `'from utils.math_util import add'`.

Q2. How does Python resolve import paths?

Answer: Python searches: (1) the current directory, (2) directories in the PYTHONPATH environment variable, (3) installation-dependent default paths stored in sys.path.

Q3. What happens when you import a module? Is it executed?

Answer: Yes. Python executes the entire module file on first import. Subsequent imports of the same module use a cached version from sys.modules without re-executing.

Q4. How can you prevent code from running when a module is imported?

Answer: Use the if `__name__ == "__main__"` guard. Only code inside this block runs when the file is executed directly.

Q5. What is sys.modules and why does it matter?

Answer: `sys.modules` is a dictionary that caches all imported modules. When you import a module, Python first checks `sys.modules`. If found, it reuses the cached module — avoiding redundant execution.

Q6. Can you reload a module without restarting Python?

Answer: Yes, using `importlib.reload(module)`. This is useful during development but should never be used in production as it can cause state inconsistencies.

Q7. What is a circular import and how do you fix it?

Answer: A circular import occurs when module A imports B and B imports A. Solutions include: refactoring shared code into a third module C, using lazy imports, or restructuring dependencies.

Q8. How do you check which functions a module provides?

Answer: Use `dir(module)` to list all attributes. Use `help(module)` for documentation. Use `module.__dict__` to see all names and their values.

Q9. What is __all__ in a module?

Answer: `__all__` is a list variable that defines which names are exported when a user does `'from module import *'`. It is a way to control the module's public API.

Q10. How are relative imports different from absolute imports?

Answer: Absolute imports specify the full path from the project root (`from utils.math_util import add`). Relative imports use dots to indicate the current or parent package (`from .math_util import add`). Absolute imports are recommended for clarity.

C. Advanced Level

Q1. Explain lazy importing and when you would use it.

Answer: Lazy importing defers the actual import until the module is first used, rather than at program startup. This reduces startup time for large applications. Implemented using `importlib.import_module()` inside a function or property. Common in large frameworks like Django for optional dependencies.

Q2. How do you create a namespace package (PEP 420)?

Answer: A namespace package is a package without an `__init__.py` file. It allows a package to span multiple directories on `sys.path`. This is useful for distributing large packages across multiple repositories or install locations.

Q3. How does Python handle module-level side effects and why are they dangerous?

Answer: Module-level code (outside functions) executes on import. This can cause database connections, file operations, or API calls to run unintentionally. This is especially dangerous in test environments. Always guard side effects with `if __name__ == '__main__':`.

Q4. How would you design a plugin system using Python modules?

Answer: Store plugin modules in a directory. At runtime, use `importlib.import_module()` to dynamically load each file. Define a common interface (e.g., a base class or a required function signature) that all plugins must implement. This is how tools like Airflow operators and pytest plugins work.

Q5. What are entry points in Python packaging and how do they relate to modules?

Answer: Entry points (defined in `setup.py` or `pyproject.toml`) expose module-level functions or classes as named hooks. They allow installed packages to advertise functionality to other packages without hardcoded imports — the foundation of plugin architectures in pip-installed tools.

12 | Data Engineer Interview Preparation

The following questions focus specifically on how module concepts appear in production data engineering environments. These reflect real questions asked in data engineering interviews at companies using Python-based data stacks.

Q1. How do you structure a PySpark ETL project using Python modules?

Answer: A professional PySpark project separates ingestion, transformation, and loading into distinct modules. A main.py entry point orchestrates the pipeline using `if __name__ == '__main__':`. Shared utilities (schema definitions, UDFs, config) live in a `utils/` package. This structure allows independent testing of each stage and clean separation of concerns.

Q2. Why do data engineers use 'import pyspark.sql.functions as F' instead of importing each function individually?

Answer: The 'F' alias is an industry-wide convention. PySpark's functions module contains hundreds of functions. Aliasing with 'F' keeps code concise (`F.col()`, `F.lit()`, `F.when()`), avoids namespace pollution, and immediately communicates to any data engineer reading the code that the functions are Spark SQL functions.

Q3. How would you manage configuration (database URLs, API keys) across modules in a pipeline?

Answer: Create a dedicated `config/settings.py` module that reads from environment variables using `os.environ` or a library like `python-dotenv`. All other modules import from `settings.py` — never hardcode credentials. This ensures secrets are managed externally (e.g., via AWS Secrets Manager or HashiCorp Vault in production) and never committed to version control.

Q4. You have a utility module that connects to a database. How do you prevent it from connecting on import?

Answer: Wrap the connection logic inside a function or class method, not at module level. Use `if __name__ == '__main__':` for any test connections. In production, use lazy initialization patterns — only establish the connection when the first query is made. Connection pooling libraries like `SQLAlchemy` handle this automatically.

Q5. What is the significance of `__init__.py` in a data engineering project?

Answer: `__init__.py` makes a directory a Python package, enabling imports like `'from ingestion.api_reader import fetch_data'`. It can also expose a clean public API by importing selected names from sub-modules, hiding internal implementation details. In large codebases, `__init__.py` files define the package's external interface.

Q6. How do Airflow DAGs use Python modules?

Answer: Each Airflow DAG file imports Python modules for task logic. Operators (`PythonOperator`, `SparkSubmitOperator`) call functions defined in separate pipeline modules. Shared utilities (connection helpers, notification functions) are packaged in a `plugins/` directory. This keeps DAG files declarative (what runs when) and separates them from procedural logic (how it runs).

Q7. How would you write a reusable data validation module for multiple pipelines?

Answer: Create a `validation/` package with modules like `schema_validator.py` and `data_quality_checks.py`. Define functions like `validate_not_null(df, columns)`, `validate_schema(df, expected_schema)`, and `check_row_count(df, min_rows)`. Any pipeline can import and run these

checks. Results can be logged or raised as exceptions. This is the foundation of data quality frameworks like Great Expectations.

Q8. What is the risk of mutable module-level state in a distributed pipeline?

Answer: In distributed systems (Spark workers, multiple Airflow task instances), each process gets its own copy of a module. Mutable module-level state (like a list or dict defined at the top of a module) is not shared between workers. If your code assumes shared state, it will behave incorrectly in distributed environments. Always use external storage (Redis, a database) for shared state.

Q9. How do you handle different environments (dev, staging, prod) in a modular pipeline?

Answer: Use environment-specific configuration files or environment variables read by `config/settings.py`. The module detects the current environment (e.g., `ENV=production`) and loads appropriate settings. Never hardcode environment-specific values in business logic modules. A common pattern is `config/dev.py`, `config/prod.py`, each imported conditionally.

Q10. Explain how you would unit test a transformation module without running the full pipeline.

Answer: Import the transformation module's functions directly in your test file. Use `pytest`. Create small in-memory DataFrames (pandas or PySpark with `SparkSession` in local mode) as test inputs. Assert that the output matches expected results. Because transformation logic is isolated in its own module, it can be tested without touching databases, APIs, or other pipeline stages.

Q11. What happens if two modules in a pipeline define a function with the same name and you use 'from module import *'?

Answer: The second import silently overwrites the first. This is a critical bug that is very hard to detect because Python gives no warning. Always use explicit imports (`import module as alias`) in production pipelines to ensure every function call is unambiguously traced to its source module.

Q12. How does Python's import caching (`sys.modules`) affect long-running pipeline processes?

Answer: In long-running processes (e.g., a Spark driver or a continuously running Airflow worker), modules are loaded once and cached in `sys.modules`. If a module file changes on disk, the running process will not pick up the changes. You must restart the process. This is why most data engineering deployments use immutable containers (Docker) — a new container image is built and deployed for every code change.

Q13. How do you share utility functions across multiple Spark jobs in a production environment?

Answer: Package utilities as a proper Python package (with `setup.py` or `pyproject.toml`). Install it in the cluster's Python environment or distribute it as a `.egg` or `.whl` file via `spark.submit.pyFiles`. This ensures every Spark worker node has access to the same utility code without copying files manually.

Q14. What is the role of `__all__` in a data engineering utility package?

Answer: `__all__` defines the public API of a module — the list of names that are exported when `'from module import *'` is used. In a utility package, it signals to other engineers which functions are stable, supported, and intended for external use, vs internal helpers. It is documentation and access control in one.

Q15. How would you implement logging in a modular pipeline?

Answer: Create a `utils/logger.py` module that configures a Python logger with appropriate handlers (console, file, or cloud logging like CloudWatch). Every other module imports: `'from utils.logger import get_logger; logger = get_logger(__name__)'`. Using `__name__` as the logger name means log messages automatically include the originating module path, making debugging across a multi-module

pipeline vastly easier.

■ Final Note for Interview Preparation

Mastering Python modules and project structure is not just a coding skill — it is the foundation of building maintainable, scalable, and production-grade data engineering systems. Every concept in today's session appears daily in real data engineering work.